Gaussian Processes

General Principles

Through varying intercepts and slopes, we have seen how to quantify some of the unique features that generate variation across clusters and covariance among the observations within each cluster. But through the covariance matrix that is used to account for correlation between clusters, we are inherently assuming linear relationships between clusters. What if we want to model the relationship between two variables that are not linearly related? In this case, we can use a Gaussian Process (GP) to model the relationship between two variables.

Considerations

Caution

- To capture complex, non-linear relationships in data where the underlying function is smooth but has an unknown functional form, GPs use a kernel .
- The choice of kernel hyperparameters can significantly impact results; thus, GPs require choosing an appropriate kernel function that captures the expected behavior of your data.
- Through kernel definition, we can incorporate domain knowledge.
- They scale poorly with dataset size ($O(n^3)$ complexity) due to matrix operations; thus, memory requirements can be substantial for large datasets, which has led to neural networks being used instead to resolve large non-linear problems.

Example

Below is an example code snippet demonstrating Gaussian Process regression using the BayesForge (BF) package. Data consist of a continuous dependent variable (*total_tools*), representing the number of tools invented in the islands, and a continuous independent variable (*population*), representing the population of the islands. The goal is to estimate the effect of population on the total tools. We use the distance matrix of the islands for the

kernel function in order to capture the spatial dependence of the relationship. This example is based on McElreath (2018).

Python

```
from BayesForge import bf
import jax.numpy as jnp
import pandas as pd
# Setup device--------------------------------------------------
m = bf(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
# Import
from importlib.resources import files
data_path = m.load.kline2(only_path=True)
m.data(data_path, sep=';')


data_path2 = files('BayesForge.Resources') / 'islandsDistMatrix.csv'
islandsDistMatrix = pd.read_csv(data_path2, index_col=0)

m.data_to_model(['total_tools', 'population'])
m.data_on_model["society"] = jnp.arange(0,10)# index observations
m.data_on_model["Dmat"] = islandsDistMatrix.values # Distance matrix

def model(Dmat, population, society, total_tools):
    a = m.dist.exponential(1, name = 'a')
    b = m.dist.exponential(1, name = 'b')
    g = m.dist.exponential(1, name = 'g')

    # non-centered Gaussian Process prior
    etasq = m.dist.exponential(2, name = 'etasq')
    rhosq = m.dist.exponential(0.5, name = 'rhosq')
    n = Dmat.shape[0]
    SIGMA = etasq * jnp.exp(-rhosq * jnp.square(Dmat))
    SIGMA = SIGMA.at[jnp.diag_indices(n)].add(0.01)
    L_chol = jnp.linalg.cholesky(SIGMA)
    z = m.dist.multivariate_normal(jnp.zeros(n), jnp.eye(n), name = 'z')
    k = jnp.matmul(L_chol, z)

    lambda_ = a * population**b / g * jnp.exp(k[society])
```

```
    m.dist.poisson(lambda_, obs=total_tools)

# Run sampler ------------------------------------------------
m.fit(model, progress_bar=False)
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a	1.40	1.05	0.24	3.39	2233.46	1746.85	1.0	0.02	0.02
b	0.28	0.09	0.15	0.42	1780.81	2000.59	1.0	0.00	0.00
etasq	0.19	0.19	0.03	0.53	1485.85	2020.54	1.0	0.00	0.01
g	0.61	0.55	0.08	1.63	1934.40	1807.15	1.0	0.01	0.01
rhosq	1.39	1.71	0.08	4.49	1647.16	1953.70	1.0	0.03	0.05
z[0]	-0.45	0.76	-1.64	0.72	2223.66	2573.74	1.0	0.02	0.01
z[1]	0.39	0.75	-0.85	1.61	3246.36	2603.44	1.0	0.01	0.01
z[2]	-0.22	0.78	-1.46	0.99	3724.48	2778.58	1.0	0.01	0.01
z[3]	0.99	0.65	0.00	2.07	2077.51	2511.39	1.0	0.01	0.01
z[4]	0.38	0.65	-0.63	1.46	2455.51	2707.30	1.0	0.01	0.01
z[5]	-1.07	0.66	-2.14	-0.04	2652.96	2559.31	1.0	0.01	0.01
z[6]	0.35	0.59	-0.60	1.28	2889.69	2717.71	1.0	0.01	0.01
z[7]	-0.42	0.69	-1.55	0.65	3567.19	2795.44	1.0	0.01	0.01
z[8]	0.76	0.63	-0.24	1.76	2504.15	2601.03	1.0	0.01	0.01
z[9]	-0.47	0.82	-1.80	0.84	2078.58	2452.71	1.0	0.02	0.01

Python (Build in function)

```
from BayesForge import bf
import jax.numpy as jnp
import pandas as pd

# Setup device-----------------------------------------------
m = bf(platform="cpu")

# Import Data & Data Manipulation ----------------------------------------------
```

```python
# Import
from importlib.resources import files

data_path = m.load.kline2(only_path=True)
m.data(data_path, sep=";")

islandsDistMatrix = m.load.islands_dist_matrix(frame=False)["data"]

m.data_to_model(["total_tools", "population"])
m.data_on_model["society"] = jnp.arange(0, 10)  # index observations
m.data_on_model["Dmat"] = islandsDistMatrix  # Distance matrix


def model(Dmat, population, society, total_tools):
    a = m.dist.exponential(1, name="a")
    b = m.dist.exponential(1, name="b")
    g = m.dist.exponential(1, name="g")

    k = m.gaussian.gaussian_process(Dmat, etasq=2, rhosq=0.5, sigmaq=0.001)

    lambda_ = a * population**b / g * jnp.exp(k[society])

    m.dist.poisson(lambda_, obs=total_tools)


# Run sampler ------------------------------------------------
m.fit(model)
m.summary()
```

```
jax.local_device_count 16

  0%|          | 0/2000 [00:00<?, ?it/s]

  0%|          | 0/2000 [00:00<?, ?it/s]

  0%|          | 0/2000 [00:00<?, ?it/s]

  0%|          | 0/2000 [00:00<?, ?it/s]
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a	1.21	1.06	0.15	3.15	761.15	638.21	1.01	0.03	0.03
b	0.31	0.14	0.08	0.53	711.82	938.45	1.00	0.01	0.00
g	0.82	0.85	0.06	2.35	800.00	784.32	1.00	0.02	0.03
kernel[0]	-0.12	0.70	-1.26	0.96	399.12	594.28	1.01	0.03	0.02
kernel[1]	0.18	0.67	-0.90	1.21	369.13	528.59	1.01	0.03	0.02
kernel[2]	0.12	0.66	-0.91	1.16	380.09	555.17	1.01	0.03	0.02
kernel[3]	0.57	0.65	-0.46	1.59	358.02	549.81	1.01	0.03	0.02
kernel[4]	0.19	0.66	-0.84	1.25	425.23	561.94	1.01	0.03	0.02
kernel[5]	-0.41	0.67	-1.46	0.65	446.40	658.03	1.01	0.03	0.02
kernel[6]	0.30	0.66	-0.74	1.35	405.26	615.67	1.01	0.03	0.02
kernel[7]	-0.17	0.68	-1.24	0.94	447.55	596.34	1.01	0.03	0.02
kernel[8]	0.41	0.67	-0.63	1.49	457.65	603.98	1.01	0.03	0.02
kernel[9]	-0.18	0.86	-1.59	1.24	533.84	728.63	1.01	0.04	0.02

R

```
library(BayesForge)
jnp = reticulate::import('jax.numpy')
pd = reticulate::import('pandas')
# setup platform-----------------------------------------------
m=importBF(platform='cpu')

# Import data -------------------------------------------------
m$data(m$load$kline2(only_path=T), sep=';')
islandsDistMatrix = m$load$islands_dist_matrix(frame = FALSE)$data
m$data_to_model(list('total_tools', 'population'))
m$data_on_model$society = jnp$arange(0,10, dtype='int64')
m$data_on_model$Dmat = jnp$array(islandsDistMatrix)


# Define model ------------------------------------------------
model <- function(Dmat, population, society, total_tools){
  a = bf.dist.exponential(1, name = 'a')
  b = bf.dist.exponential(1, name = 'b')
  g = bf.dist.exponential(1, name = 'g')

  # non-centered Gaussian Process prior
  etasq = bf.dist.exponential(2, name = 'etasq')
  rhosq = bf.dist.exponential(0.5, name = 'rhosq')
```

```
  k = m$gaussian$gaussian_process(Dmat, etasq, rhosq, 0.001)

  lambda_ = a * population**b / g * jnp$exp(k[society])
  m$dist$poisson(lambda_, obs = total_tools, name = "y")
}

# Run MCMC ------------------------------------------------
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary -------------------------------------------------
m$summary() # Get posterior distribution
```

Julia

```
using PyBayesForge

# Setup device--------------------------------------------------
m = importBF(platform="cpu")

# Import Data & Data Manipulation ------------------------------------------------
# Import
data_path = m.load.kline2(only_path = true)
m.data(data_path, sep=";")

islandsDistMatrix = m.load.islands_dist_matrix(frame = false)["data"]
m.data_to_model(["total_tools", "population"])
m.data_on_model["society"] = jnp.arange(0,10)# index observations
m.data_on_model["Dmat"] = jnp.array(islandsDistMatrix) # Distance matrix



# Define model -------------------------------------------------
@BF function model(Dmat, population, society, total_tools)
    a = m.dist.exponential(1, name = "a")
    b = m.dist.exponential(1, name = "b")
    g = m.dist.exponential(1, name = "g")

    # non-centered Gaussian Process prior
    etasq = m.dist.exponential(2, name = "etasq")
    rhosq = m.dist.exponential(0.5, name = "rhosq")
    n = Dmat.shape[0]
```

```
    SIGMA = etasq * jnp.exp(jnp.negative(rhosq) * jnp.square(Dmat))
    SIGMA = SIGMA.at[jnp.diag_indices(n)].add(0.01)
    L_chol = jnp.linalg.cholesky(SIGMA)
    z = m.dist.multivariate_normal(jnp.zeros(n), jnp.eye(n), name = "z")
    k = jnp.matmul(L_chol, z)

    lambda_ = a * population^b / g * jnp.exp(k[society])

    m.dist.poisson(lambda_, obs=total_tools, name="y")

end

# Run mcmc ------------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary -------------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

Formula

The following equation allows us to evaluate the relationship between the dependent variable Y distributed normal, and the independent variable X while incorporating a GP for the effect of variable Q:

$$Y_{[i]} \sim \text{Normal}(\alpha + \beta X_{[i]} + \gamma_{[Q_{[i]}]}, \sigma)$$

where:

- $Y_{[i]}$ is the i-th value for the dependent variable Y.
- α is the intercept term.
- β is the regression coefficient term.
- $X_{[i]}$ is the i-th value for the independent variable X.
- $Q_{[i]}$ is an integer-valued independent variable (e.g., year-of-birth, age, year) for observation i.
- γ is a vector output from a Gaussian process:

$$\gamma \sim \text{MVNormal}\left(Z, \varsigma \Omega \varsigma\right)$$

where:

- Z represents the mean vector of the multivariate normal distribution and set to zero .
- ς is a diagonal matrix of standard deviations.
- Ω is a correlation matrix.
- Multiple kernel functions for Ω exist and will be discussed in the Note(s) section. But the most common one is the quadratic kernel:

$$\Omega_{[i,j]} = \eta \exp(-\phi^2 D^2_{[i,j]})$$

Where:

- η is the maximal correlation.
- ϕ determines the rate of decline.
- $D_{[i,j]}$ is the distance between the i-th and j-th categories.

Bayesian model

In the Bayesian formulation, we define each parameter with priors . We can express a Bayesian version of this GP using the following model:

$$Y_i = \alpha + \beta X_i + \gamma_{Z_i}$$

$$\gamma \sim \text{MVNormal}\left(\begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}, K\right)$$

$$K_{ij} = \eta^2 \exp(-p^2 D^2_{ij}) + \delta_{ij}\sigma^2$$

$$\alpha \sim \text{Normal}(0, 1)$$

$$\eta^2 \sim \text{HalfCauchy}(0, 1)$$

$$p^2 \sim \text{HalfCauchy}(0, 1)$$

where:

- Y_i is the i-th value for the dependent variable Y.
- α is the intercept term with a prior of Normal$(0, 1)$.
- β is the regression coefficient term with a prior of Normal$(0, 1)$.
- X_i is the i-th value for the independent variable X.
- γ_{Z_i} is the Gaussian process i-th value for the independent variable Z.
- γ is the latent function modeled by the GP.
- K_{ij} is the kernel function evaluated at the corresponding points, $K_{ij} = k(Z_i, Z_j)$, with priors of HalfCauchy(0,1) for η^2 and p^2 to ensure positive values.

Model validation tests

- Comparison with Stan: `13.Gaussian processes.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence).
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> Common kernel functions include:
>
> - *Radial Basis Function* (RBF) or Squared Exponential Kernel:
>
> $$k(x, x') = \sigma^2 \exp\left(-\frac{||x - x'||^2}{2l^2}\right)$$
>
> - *Rational Quadratic Kernel*, this kernel is equivalent to adding together many RBF kernels with different length scales:
>
> $$k(x, x') = \sigma^2 \left(1 + \frac{||x - x'||^2}{2l^2}\right)^{-\alpha}$$

- *Periodic kernel* allows for modeling functions that repeat themselves exactly:

$$k(x, x') = \sigma^2 \exp\left(-\frac{2\sin^2(\pi||x - x'||/p)}{l^2}\right)$$

- *Locally Periodic Kernel*:

$$k(x, x') = \sigma^2 \exp\left(-\frac{2\sin^2(\pi||x - x'||/p)}{l^2}\right) \exp\left(-\frac{||x - x'||^2}{2l^2}\right)$$

- Any slope or intercept in your model can be defined using a Gaussian Process.

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.

https://www.cs.toronto.edu/~duvenaud/cookbook/